

# Unit 1

## Unit 1: HTML, CSS, Bootstrap & W3C — Practical/Oral Exam Notes

### Context

Part of Web Application Development | Practical + Oral Exam Prep | SPPU Third Year IT

### High-Yield Oral Topics

- HTML5 Semantic Elements** — Examiners love this. Know 6+ examples.
- CSS Flexbox vs Grid** — Always compared in viva.
- Bootstrap Grid (col-md-6)** — Must explain responsiveness.
- CSS Box Model** — Margin, Border, Padding, Content.
- Inline vs Block vs Inline-Block** — Trap question.

## MINI PROJECT ANCHOR: Student Management System (SMS)

All examples in this unit use the **Student Management System** for consistency.

- Frontend: HTML + CSS + Bootstrap
- Features: Student Registration Form, Dashboard, Student List Table

## SECTION 1 — HTML

### Exam Definition

HTML (HyperText Markup Language) is the **standard markup language** used to create the structure and content of web pages. It uses **tags** to define elements like headings, paragraphs, forms, and images.

### Core Concept

HTML is NOT a programming language — it is a **markup language**. It tells the browser *what* to display, not *how* to behave. CSS handles appearance; JavaScript handles behavior.

### Must-Know Tags (Viva Favorite)

| Category  | Tags                     | Purpose       |
|-----------|--------------------------|---------------|
| Structure | <html> , <head> , <body> | Page skeleton |

| Category | Tags                                                | Purpose              |
|----------|-----------------------------------------------------|----------------------|
| Headings | <h1> to <h6>                                        | Hierarchy of text    |
| Text     | <p> , <span> , <strong> , <em>                      | Content              |
| Links    | <a href="">                                         | Navigation           |
| Lists    | <ul> , <ol> , <li>                                  | Ordered/Unordered    |
| Tables   | <table> , <tr> , <th> , <td>                        | Tabular data         |
| Forms    | <form> , <input> , <button> , <select>              | User input           |
| Media    | <img> , <video> , <audio>                           | Rich media           |
| Semantic | <header> , <nav> , <section> , <article> , <footer> | Meaningful structure |

## HTML Forms and Structure

Instead of viewing full code, understand the theoretical composition of a form based on the syllabus:

- `<form>` : The container for input elements. The `action` attribute specifies where to send data, and `method` (GET/POST) specifies how.
- `<input>` : The most important form element. The `type` attribute determines if it's text, email, password, date, etc.
- `<select>` : Creates a drop-down list.
- `<label>` : Defines a label for a form element, improving accessibility. Use the `for` attribute.

## Semantic HTML5 — MOST ASKED IN VIVA

| Semantic Tag | Purpose                            | Non-Semantic Equivalent |
|--------------|------------------------------------|-------------------------|
| <header>     | Top section of page/section        | <div id="header">       |
| <nav>        | Navigation links                   | <div id="nav">          |
| <main>       | Main content of page               | <div id="main">         |
| <section>    | Thematic grouping of content       | <div>                   |
| <article>    | Self-contained content (blog post) | <div>                   |
| <aside>      | Sidebar / related content          | <div id="sidebar">      |
| <footer>     | Bottom of page                     | <div id="footer">       |
| <figure>     | Image with caption                 | <div>                   |

**Why Semantic HTML?** — SEO (Search Engine Optimization), Accessibility (screen readers), Code readability.

## HTML Attributes — Viva Quick Fire

| Attribute | Used On          | Purpose                        |
|-----------|------------------|--------------------------------|
| href      | <a>              | Link destination               |
| src       | <img> , <script> | File source                    |
| alt       | <img>            | Alternate text (accessibility) |

| Attribute | Used On | Purpose                           |
|-----------|---------|-----------------------------------|
| action    | <form>  | Where to send form data           |
| method    | <form>  | GET or POST                       |
| type      | <input> | text, email, password, date, etc. |
| required  | <input> | HTML5 validation                  |
| id        | Any     | Unique identifier                 |
| class     | Any     | CSS styling hook                  |

## VIVA Q&A — HTML

**Q1: What is the difference between <div> and <span> ?**

`<div>` is a **block-level** element — takes full width, starts on new line. Used for layout sections. `<span>` is an **inline** element — only as wide as its content. Used to style a portion of text.

**Q2: What is the difference between GET and POST in forms?**

**GET** appends data to URL (visible, used for search). **POST** sends data in request body (hidden, secure, used for login/registration). POST is preferred for sensitive data like passwords.

**Q3: What are HTML5 Semantic elements and why use them?**

HTML5 semantic elements like `<header>`, `<nav>`, `<article>` give **meaning** to the structure. Benefits: Better SEO (Google understands page structure), Accessibility (screen readers), and cleaner code.

**Q4: What is the DOCTYPE declaration?**

`<!DOCTYPE html>` tells the browser to render the page in **HTML5 standards mode**. Without it, browsers go into "quirks mode" which causes inconsistent rendering.

**Q5: What is alt attribute in <img> ?**

The `alt` attribute provides **alternative text** that displays if the image fails to load. It is also used by **screen readers** for visually impaired users and improves **SEO**.

## Common Mistakes Students Make

- Forgetting `<!DOCTYPE html>` at top
- Not closing tags (especially `<li>`, `<td>` )
- Using `<b>` instead of `<strong>` (semantic vs presentational)
- Not adding `name` attribute to form inputs (data won't be sent)
- Mixing up `id` (unique) vs `class` (reusable)

## Memory Trick

"Semantic HTML = You Know What's Inside Without Opening the Box"

`<article>` → article content. `<nav>` → navigation. `<footer>` → bottom info.

## SECTION 2 — CSS

### 📌 Exam Definition

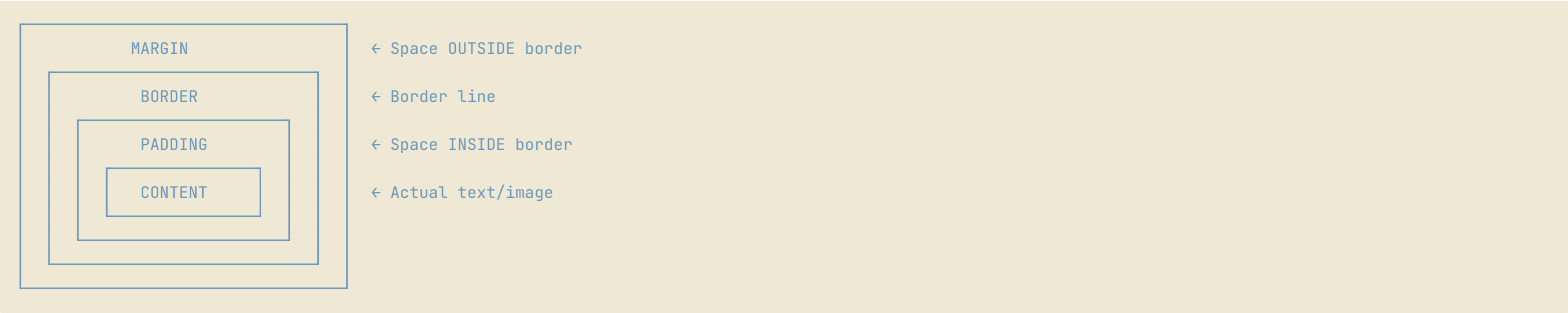
CSS (Cascading Style Sheets) is a **stylesheet language** used to control the presentation (layout, colors, fonts, spacing) of HTML elements. It **separates design from structure**.

### 🔑 Three Types of CSS

| Type     | Syntax                                                      | Priority | Use Case                |
|----------|-------------------------------------------------------------|----------|-------------------------|
| Inline   | <code>style="color:red"</code> inside tag                   | Highest  | Quick one-off styles    |
| Internal | <code>&lt;style&gt;</code> inside <code>&lt;head&gt;</code> | Medium   | Single-page styles      |
| External | <code>&lt;link rel="stylesheet" href="style.css"&gt;</code> | Normal   | Best practice, reusable |

📌 **Cascade Rule:** Inline > Internal > External (when specificity is equal)

### 📦 The CSS Box Model — MUST KNOW



📌 **Viva Answer:** "Margin is OUTSIDE the border, Padding is INSIDE the border. Margin creates space between elements; padding creates space between content and its border."

### 📌 Core CSS Properties (Syllabus Highlights)

Instead of a long stylesheet, here are the essential properties you need to know for the exam:

- **Box Model Properties:** `margin` (outer space), `padding` (inner space), `border` (boundary), and `width` / `height`.
- **Positioning:** `static`, `relative`, `absolute`, `fixed`, `sticky`.
- **Layouts:**

- **Flexbox:** `display: flex;` for 1D alignment (rows or columns). Key properties: `justify-content`, `align-items`.
- **Grid:** `display: grid;` for 2D layouts. Key properties: `grid-template-columns`, `gap`.
- **Transitions & Animations:**
  - `transition: property duration timing-function;` for smooth changes.
  - `@keyframes` combined with the `animation` property for complex movements.

## 🔑 Flexbox vs Grid — MOST COMPARED IN VIVA

| Feature   | Flexbox                                                 | Grid                                                  |
|-----------|---------------------------------------------------------|-------------------------------------------------------|
| Dimension | 1-Dimensional (row OR column)                           | 2-Dimensional (rows AND columns)                      |
| Use Case  | Navigation bars, card rows, centering                   | Page layout, dashboards                               |
| Direction | <code>flex-direction: row/column</code>                 | <code>grid-template-columns/rows</code>               |
| Alignment | <code>justify-content</code> , <code>align-items</code> | <code>justify-items</code> , <code>align-items</code> |
| Example   | Navigation links in a row                               | 3-column dashboard                                    |

## CSS Positioning — Viva Trap

| Value                 | Behavior                                                             |
|-----------------------|----------------------------------------------------------------------|
| <code>static</code>   | Default. Normal flow. <code>top/left</code> ignored.                 |
| <code>relative</code> | Offset from its NORMAL position. Other elements not affected.        |
| <code>absolute</code> | Removed from flow. Positioned relative to nearest positioned parent. |
| <code>fixed</code>    | Removed from flow. Fixed to browser viewport (sticky navbar).        |
| <code>sticky</code>   | Normal flow until scroll threshold, then fixed.                      |

## 🎤 VIVA Q&A — CSS

### Q1: What is specificity in CSS?

Specificity determines which CSS rule applies when multiple rules target the same element. Priority: Inline style (1000) > ID selector (100) > Class selector (10) > Element selector (1). Higher specificity wins.

### Q2: What is the difference between `margin` and `padding`?

Margin is the space **outside** the border (between elements). Padding is the space **inside** the border (between content and border). Background color fills the padding area but NOT the margin.

### Q3: What is `box-sizing: border-box`?

By default, width/height only includes content. With `border-box`, width/height includes padding and border too. This makes layout calculation much easier and predictable.

### Q4: How does CSS animation work?

CSS animation uses `@keyframes` to define style changes at different points, and `animation` property applies it to an element. Example: `animation: fadeIn 0.5s ease` – the element fades in over 0.5 seconds.

### 💡 Memory Tricks

- **Box Model:** "My Big Precious Content" = Margin, Border, Padding, Content (outside to inside)
- **Flex is 1D, Grid is 2D** → Flex = one direction at a time, Grid = full table layout

## SECTION 3 — BOOTSTRAP

### 📌 Exam Definition

Bootstrap is a **free, open-source CSS framework** that provides pre-built responsive components and a **12-column grid system**, allowing developers to build mobile-responsive web pages rapidly without writing CSS from scratch.

### Why Bootstrap over Raw CSS?

| Aspect                | Raw CSS                       | Bootstrap                  |
|-----------------------|-------------------------------|----------------------------|
| Time                  | Write everything from scratch | Pre-built components ready |
| Responsiveness        | Manual media queries          | Built-in grid system       |
| Browser Compatibility | Test/fix manually             | Handled by Bootstrap       |
| Consistency           | Depends on developer          | Consistent across team     |
| Learning Curve        | Deep CSS knowledge needed     | Learn classes, apply fast  |

### 🔑 Bootstrap Grid System — MUST KNOW

Bootstrap divides every row into **12 columns**. You specify how many columns an element takes at each breakpoint.

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Left Sidebar</div>
    <div class="col-md-8">Main Content</div>
  </div>
</div>
```

### Bootstrap Breakpoints

| Breakpoint  | Code                 | Width   | Target       |
|-------------|----------------------|---------|--------------|
| Extra Small | <code>col-</code>    | < 576px | Phones       |
| Small       | <code>col-sm-</code> | ≥ 576px | Large phones |



| Breakpoint  | Code    | Width    | Target   |
|-------------|---------|----------|----------|
| Medium      | col-md- | ≥ 768px  | Tablets  |
| Large       | col-lg- | ≥ 992px  | Laptops  |
| Extra Large | col-xl- | ≥ 1200px | Desktops |

**Viva Tip:** col-md-6 means "take 6 of 12 columns on medium screens and above" = **two equal columns** on tablet/desktop.

## Key Bootstrap Classes — Cheatsheet

| Class                         | Purpose                               |
|-------------------------------|---------------------------------------|
| container / container-fluid   | Centered wrapper / Full-width wrapper |
| row                           | Flex row (parent of columns)          |
| col-md-6                      | Column taking 6/12 width on medium+   |
| btn btn-primary               | Blue button                           |
| table table-striped           | Striped table                         |
| card, card-body, card-title   | Card component                        |
| navbar navbar-expand-lg       | Responsive navigation bar             |
| d-flex justify-content-center | Flexbox centering                     |
| mt-3, mb-4, px-2              | Margin/Padding utilities              |
| text-center, fw-bold          | Text alignment/weight                 |
| d-none d-md-block             | Hide on mobile, show on desktop       |

## VIVA Q&A — Bootstrap

### Q1: What is the Bootstrap grid system?

Bootstrap's grid system uses a series of containers, rows, and columns to layout content. It's based on a **12-column** flexible grid. You specify how many columns an element spans using classes like col-md-6 (6 of 12 = half width on medium+ screens).

### Q2: How does Bootstrap achieve responsiveness?

Bootstrap uses **CSS media queries** internally. When you use col-md-6 col-12, Bootstrap applies display: block; width: 100% on small screens and width: 50% on medium+ screens automatically.

### Q3: What is the difference between container and container-fluid ?

container has a fixed max-width that changes at breakpoints (centered with margins). container-fluid is always 100% wide — it spans the full viewport width.

### Q4: Why use Bootstrap instead of writing CSS from scratch?

Bootstrap saves development time with pre-built components (buttons, navbars, modals, tables), ensures cross-browser compatibility, and provides a proven responsive grid system. It's ideal for rapid prototyping and team projects.

## Oral Trap Questions

**Trap: "Can you use Bootstrap AND custom CSS together?"**

Yes! You add Bootstrap via CDN, then write your own CSS file after it. Your custom styles override Bootstrap where needed. This is the standard approach.

**Trap: "Bootstrap uses JavaScript — what for?"**

Bootstrap includes JavaScript (via Popper.js) for **interactive components** like modals, dropdowns, carousels, tooltips, and collapsible navbars. Pure CSS cannot handle these interactions.

## SECTION 4 — W3C

### Exam Definition

W3C (World Wide Web Consortium) is the **international standards organization** that develops and maintains open web standards to ensure long-term growth and interoperability of the web.

### Key Points for Viva

| Aspect        | Details                                                |
|---------------|--------------------------------------------------------|
| Founded by    | Tim Berners-Lee (inventor of WWW) in 1994              |
| Purpose       | Set standards for HTML, CSS, XML, Web Accessibility    |
| Key Standards | HTML5, CSS3, WCAG (Accessibility), SVG, WebRTC         |
| Why Important | Ensures all browsers render the same code consistently |

### VIVA Q&A — W3C

**Q1: What is W3C and why is it important?**

W3C (World Wide Web Consortium) is the body that creates and maintains web standards. Without W3C, each browser would implement HTML and CSS differently, breaking cross-browser compatibility. W3C ensures that code written by one developer works consistently everywhere.

**Q2: What is WCAG?**

WCAG (Web Content Accessibility Guidelines) is a W3C standard that defines how to make web content accessible to people with disabilities (visual, auditory, cognitive). It defines criteria like sufficient color contrast and alt text for images.

## PRACTICAL FILE EXPECTATIONS — Assignment 1

Your practical file for Assignment 1 should show:

- HTML Registration Form** — Student registration with name, email, course, DOB fields



- 2. **Bootstrap Dashboard** — Sidebar + 3 statistics cards + student table
- 3. **CSS Styling** — External stylesheet with responsive design
- 4. **JavaScript Validation** — Form validation + store data in `localStorage`
- 5. **AJAX POST** — Submit form data via AJAX and display success message
- 6. **Student List Page** — Read from localStorage and display in table

Viva Flow for Assignment 1:

"I created a Student Management System frontend. The registration page uses **Bootstrap grid** (col-md-3 for sidebar, col-md-9 for content). The form uses **HTML5 validation** (required, type="email"). On submit, **JavaScript** validates the data and stores it in **localStorage** as a JSON array. I also used **AJAX POST** to send the data to the backend. The dashboard shows student statistics using **Bootstrap cards**."

🔥 High Probability Exam Questions

- 1. What is the difference between HTML and HTML5? (Answer: HTML5 added semantic elements, audio/video support, canvas, local storage, form validation)
- 2. Explain the CSS Box Model with diagram
- 3. What is Flexbox? How is it different from Grid?
- 4. What is Bootstrap Grid? Explain with `col-md-6` example
- 5. What are semantic elements? Give 5 examples
- 6. Difference between `margin` and `padding`
- 7. What is media query? How does it work?
- 8. What is specificity in CSS?
- 9. What is `DOCTYPE` ?
- 10. What is the difference between `id` and `class` in CSS?

🧠 Quick Revision Table

| Topic         | Key Point                      | Viva One-Liner                                    |
|---------------|--------------------------------|---------------------------------------------------|
| HTML          | Markup language for structure  | "HTML tells browser WHAT to show"                 |
| CSS           | Stylesheet for presentation    | "CSS tells browser HOW to show it"                |
| Bootstrap     | CSS framework with 12-col grid | "Responsive design without writing media queries" |
| Semantic HTML | Tags with meaning              | "Screen readers and SEO benefit"                  |
| Flexbox       | 1D layout                      | "Row OR column alignment"                         |
| Grid          | 2D layout                      | "Rows AND columns layout"                         |
| Box Model     | M-B-P-C                        | "Margin outside, Padding inside border"           |
| W3C           | Standards body                 | "Tim Berners-Lee's web standards org"             |